Implementation procedure: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

Karol Daniela Correa

November 8, 2026

Abstract

This document describes the technical procedure for the implementation, deployment, and startup of the FaceAttendEDU system. It details the deployment strategy based on containerization using Docker and orchestration with Docker Compose, ensuring the correct configuration of the data infrastructure (PostgreSQL, MongoDB, Redis, and Apache Kafka) and the deployment of polyglot microservices (Docker Inc., n.d.). The document provides a step-by-step guide covering from the minimum hardware and software requirements to the final verification protocol, ensuring that the operational environment is consistent, secure, and traceable.

Keywords: software implementation, microservices deployment, Docker, Docker Compose, data infrastructure, environment configuration

Contents

Introduction 4
Document Objective 4
Document Scope 4
Implementation Strategy 4
Deployment Model: Containerization 4
Implementation Requirements 5
Hardware Requirements (Minimum) 5
Software Requirements 5
Installation Procedure 5
1. Environment Preparation 5
2. Obtaining the Project 6
3. System Configuration 6
4. Deployment and Orchestration 6
Verification of the Implementation 6
Operation and Maintenance 7
Frequent Commands 7
Common Troubleshooting 7
Conclusions 8
References 8
Deployment evidence 9
Production environment 9
Latest deployment 9
Production check 9

Implementation procedure: FaceAttend EDU

Introduction

The software implementation is the phase where the architectural design and the developed code are transformed into an operational and accessible system for users. For FaceAttendEDU, given the complexity of having nine microservices implemented in four different languages (Java, TypeScript, Python, and Go), a container-based deployment strategy has been adopted.

This approach eliminates the need to manually install each runtime environment on the host machine, encapsulating all dependencies within standardized images. This ensures that the system behaves identically in development, testing, and production environments.

Document Objective

To establish the formal and detailed procedure for the installation, configuration, and deployment of the FaceAttendEDU system, allowing any qualified technician to put the platform into operation without errors and in a reproducible manner.

Document Scope

The document covers the environment preparation, base tool installation, environment variable configuration, data infrastructure deployment, microservices orchestration, and the system health verification protocol. It does not include external network configuration (DNS, corporate Firewall) or high availability configuration in the cloud.

Implementation Strategy

Deployment Model: Containerization

The system utilizes *Docker* as the container platform and *Docker Compose* as the orchestrator. This choice allows the management of the 25 containers necessary (16 long-running services, databases, and gateway, plus 9 one-off migration tasks) through a single configuration file.

Network Organization

To ensure security, the deployment divides traffic into three isolated networks:

1. *Edge Network*: Connects the web interface and the Gateway (Kong) (Kong Inc., n.d.).
2. *App Network*: Connects the Gateway with the microservices.
3. *Data Network*: Connects the microservices with the databases and the messaging broker.

This architecture prevents the presentation layer from having direct access to the data layer, mitigating the risk of direct attacks on the database.

Implementation Requirements

Hardware Requirements (Minimum)

Table 1

Hardware Requirements (Minimum)

Resource	Requirement	Justification
RAM	8 GB (16 GB Rec.)	Simultaneous execution of multiple JVMs and data engines.
CPU	4 Cores (x86_64/ARM)	Biometric processing and container orchestration.
Disk	20 GB free	Storage for Docker images and data volumes.
Network	Internet access	Download of base images and dependencies during the first build.

Source. Own elaboration.

Software Requirements

- *Operating System*: Windows 10/11 (with WSL2), Ubuntu 22.04 LTS, or macOS Ventura+.
- *Docker*: Version 24.0 or higher.
- *Docker Compose*: Version 2.20 or higher.
- *Git*: Version 2.40 or higher for obtaining the source code.

Installation Procedure

1. Environment Preparation

Before starting, the installer must verify the availability of critical ports (e.g., 8000 for the Gateway, 5432 for PostgreSQL). In case of conflict, local services must be stopped or the configuration modified.

2. Obtaining the Project

The source code is obtained by cloning the official repository:

```
git clone https://github.com/Jonas7891/ProyectoFaceAttendEDU.git
cd ProyectoFaceAttendEDU/FULL
```

3. System Configuration

The system configuration is centralized in a `.env` file at the project root.

1. Copy the `.env.example` file to `.env`.
2. Adjust variables according to the environment (especially `BIND_IP` and `EXPO_PUBLIC_API_URL` if accessed from mobile devices).

4. Deployment and Orchestration

The deployment is executed with the following command, which builds the images and launches the containers in the background:

```
docker compose up -d --build
```

Startup Order and Migrations

The system follows a strict dependency chain:

1. *Data Layer*: PostgreSQL, MongoDB, Redis, and Kafka start.
2. *Migrations*: Eight sequential *Liquibase* tasks run to create schemas and load master data (Liquibase, n.d.).
3. *Microservices*: Once migrations finish, the business services start.
4. *Gateway and Interface*: Finally, the Gateway (Kong) and the web interface are launched.

Verification of the Implementation

To declare the installation as successful, the following test protocol must be executed:

Table 2

Verification of the Implementation

Test	Verification Method	Expected Result
Container Status	`docker compose ps -a`	16 containers running (healthy) and 9 one-off tasks exited with code 0.
Service Health	`curl http://localhost:8081/api/v1/health` (one port per service)	JSON response with status "UP".
Gateway Connectivity	`curl -i http://localhost:8000/api/v1/auth/session`	401 from the Kong Gateway: the route requires a session.
Authentication	Login with the first administrator created by `bootstrap-admin.sh`	Return of a valid session token.
Interface Loading	Access `http://localhost:8090`	Correct loading of the login screen.
Network Isolation	`docker exec ... nslookup postgres`	Resolution failure from the frontend.

Source. Own elaboration.

Operation and Maintenance

Frequent Commands

- *Restart system*: `docker compose restart`
- *View logs in real-time*: `docker compose logs -f <service>`
- *Update version*: `git pull` → `docker compose up -d --build`
- *Total cleanup (including data)*: `docker compose down -v`

Common Troubleshooting

- *Port Occupied*: Identify the process with `netstat -ano` and terminate it or change the port in the `.env`.
- *Migration Error*: Review the `identity-liquibase` logs to detect syntax or connectivity failures.
- *Interface not connecting*: Verify that `EXPO_PUBLIC_API_URL` matches the host IP and that the frontend has been rebuilt (`--build`).

Conclusions

The implementation procedure for FaceAttendEDU minimizes manual intervention through Docker automation. The separation of networks and dependent orchestration ensure that the system is deployed consistently, drastically reducing "works on my machine" issues and deployment time.

Author's Note. Jonattan Steven Rizo Solano is the technical lead for the implementation strategy. Technical correspondence can be directed to jonas@consultoria.example.

References

Docker Inc. (n.d.). *Docker documentation*. https://docs.docker.com/

Kong Inc. (n.d.). *Kong Gateway documentation*. https://docs.konghq.com/gateway/

Liquibase. (n.d.). *Liquibase documentation*. https://docs.liquibase.com/

Deployment evidence

Production environment

Table 3

Production environment

Component	Platform	Address
Web application	Vercel	https://faceattend-frontend.vercel.app
API, through Kong	AWS EC2	`https://faceattend-edu.duckdns.org`
Relational database	Neon (PostgreSQL)	The connection is not published

Source. Own elaboration.

Latest deployment

- *Version:* `v1.7.0`, commit `07119ab7`.
- *Date:* November 4, 2026. Times are in UTC.
- *Result:* all stages finished green.
- *Evidence:* Deploy production run.

Table 4

Latest deployment

Stage	Jobs	Latest completion (UTC)	Result
CI	13	21:09:59	Success
database / liquibase / neon / production	1	21:13:18	Success
backend / ec2 / production	1	21:18:49	Success
frontend / vercel / production	1	21:25:52	Success

Source. Own elaboration.

Production check

Check made on November 4, 2026, without signing in:

Table 5

Production check

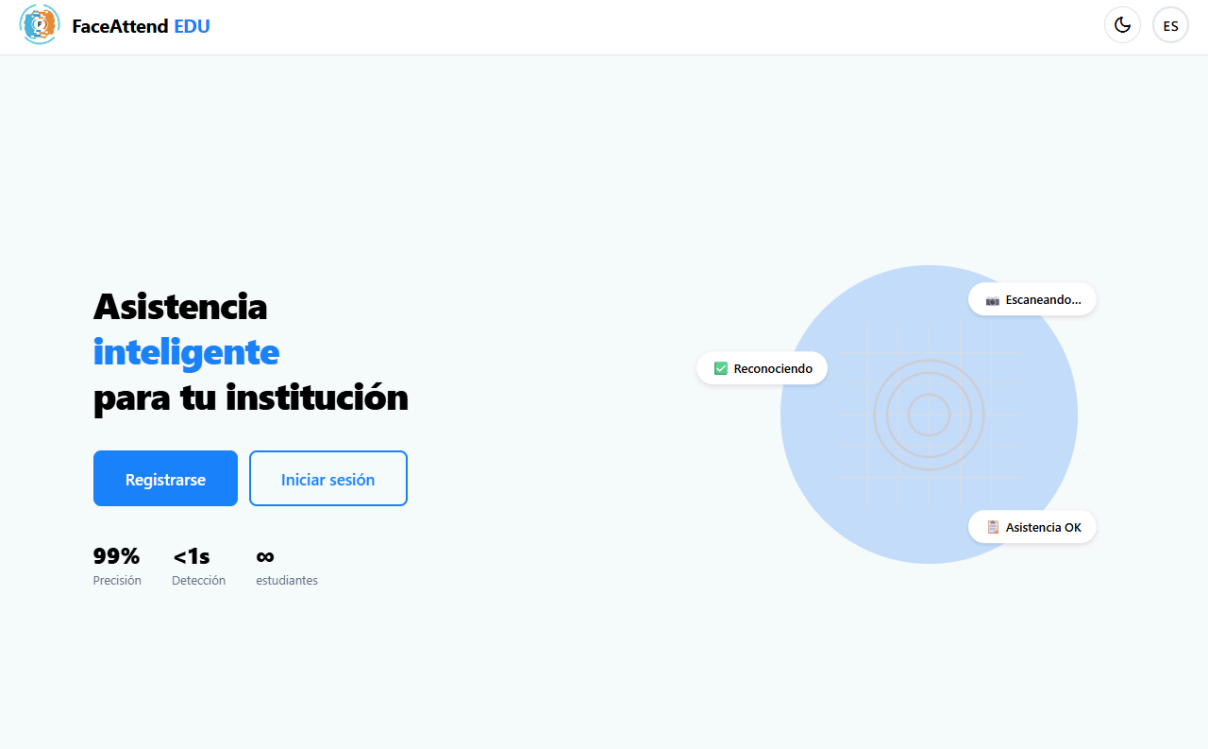
Check	Response	Meaning
`GET https://faceattend-frontend.vercel.app/`	200	The web application is published
`GET https://faceattend-edu.duckdns.org/api/v1/auth/session`	401	The gateway responds and requires a session

Source. Own elaboration.

The following image shows the public screen of the web application in production.

Figure 1

Public screen of the web application in production



Source. Own elaboration.